Practical1:- Menu-Driven Program for Array Manipulations

Problem Statement: Program to Create and insert & Display an Array

```
#include<iostream>
using namespace std;

int main() {
    int arr[100], n, pos, value;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Enter position to insert (0 to " << n << "): ";
    cin >> pos;

    cout << "Enter value to insert: ";
    cin >> value;

    // shift elements to the right
    for (int i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }

    arr[pos] = value;
    n++;

    cout << "Array after insertion:\n";
    for (int i = 0; i < n; i++) {
        cout << arr[i]<<" ";
    }

    return 0;
}
```

Practical 2 :- Menu-Driven Program for Student Performance Analysis

```
#include<iostream>
using namespace std;

int main()
{
   int roll;
   float m1, m2, m3, m4, m5;
   float total, percentage;

cout << "Enter Roll Number: ";
cin >> roll;

cout << "Enter marks of 5 subjects: ";
cin >> m1 >> m2 >> m3 >> m4 >> m5;

total = m1 + m2 + m3 + m4 + m5;
percentage = total / 5;

cout << "\nTotal Marks = " << total;
cout << "\nPercentage = " << percentage << "%";

   if(percentage >= 75)
      cout << "\nResult: Distinction";
   else if(percentage >= 60)
      cout << "\nResult: First Class";
   else if(percentage >= 50)
      cout << "\nResult: Second Class";
   else if(percentage >= 40)
      cout << "\nResult: Pass";
   else
      cout << "\nResult: Fail";

   return 0;
}
```

Practical 3:

Searching an Element in an Array Using Linear Search Technique”

Implementation of Linear Search Algorithm in C++ linear search

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n, key, i;
    int arr[50];

    cout << "Enter number of elements: ";
    cin >> n;

cout << "Enter elements of array:\n";
for (i = 0; i < n; i++) {
    cin >> arr[i];
}

cout << "Enter element to search: ";
cin >> key;

for (i = 0; i < n; i++) {
    if (arr[i] == key) {
        cout << "Element found at position " << i + 1;
        return 0;
    }
}

    cout << "Element not found";
    return 0;
}
```

Practical 4:- Title: Sorting Product Prices using Bubble Sort for an E-commerce Application

```
#include<iostream>
using namespace std;

int main() {
    int n;
    float price[50], temp;

    cout << "Enter number of products: ";
    cin >> n;

    cout << "Enter product prices:\n";
```

```
for (int i = 0; i < n; i++) {
    cin >> price[i];
}

// Bubble Sort
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (price[j] > price[j + 1]) {
            temp = price[j];
            price[j] = price[j + 1];
            price[j + 1] = temp;
        }
    }
}

cout << "\nSorted Product Prices (Ascending Order):\n";
for (int i = 0; i < n; i++) {
    cout << price[i]<<" ";
}

    return 0;
}
```

Practical 5:- Title: Singly Linked List Operations

Problem Statement:

write program in C++ using singly linked list

```
#include<iostream>
using namespace std;

// Node structure
struct Node {
    int data;
    Node* next;
};

Node* head = NULL;

// Insert node at end
void insertEnd(int value) {
    Node* newNode = new Node();
    newNode->data = value;
```

```
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

// Display linked list
void display() {
    Node* temp = head;
    if (temp == NULL) {
        cout << "List is empty";
        return;
    }
    cout << "Linked List: ";
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL";
}

int main() {
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);

    display();

    return 0;
}
```

Practical 6:- Circular Linked List-Based Token Management in Service Centers

```
#include<iostream>
using namespace std;

// Node structure
struct Node {
    int data;
    Node* next;
};

Node* head = NULL;

// Insert node at end
void insertEnd(int value) {
    Node* newNode = new Node();
    newNode->data = value;

    if (head == NULL) {
        head = newNode;
        newNode->next = head;   // circular link
    } else {
        Node* temp = head;
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;   // circular link
    }
}

// Display circular linked list
void display() {
    if (head == NULL) {
    cout << "List is empty";
    return;
}

Node* temp = head;
cout << "Circular Linked List: ";
do {
    cout << temp->data << " -> ";
```

```
        temp = temp->next;
    } while (temp != head);
    cout << "(back to head)";
}

int main() {
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);

    display();

    return 0;
}
```

Practical 7:- Stack Implementation Using Arrays
Problem Statement: Write a menu-driven program to implement a stack as an ADT using arrays. Include operations such as push, pop, peek (top element), and display. Ensure overflow and underflow conditions are handled. Use this ADT for decimal to binary conversion using stack.

```
#include<iostream>
using namespace std;

#define MAX 5

int stack[MAX];
int top = -1;

// Push operation
void push(int value) {
    if (top == MAX - 1) {
        cout << "Stack Overflow\n";
        return;
    }
    top++;
    stack[top] = value;
    cout << value << " pushed into stack\n";
}

// Pop operation
void pop() {
```

```
    if (top == -1) {
        cout << "Stack Underflow\n";
        return;
    }
    cout << stack[top] << " popped from stack\n";
    top--;
}

// Display stack
void display() {
    if (top == -1) {
        cout << "Stack is empty\n";
        return;
    }
    cout << "Stack elements:\n";
    for (int i = top; i >= 0; i--) {
        cout << stack[i] << endl;
    }
}

int main() {
    push(10);
    push(20);
    push(30);

    display();

    pop();
    display();

    return 0;
}
//stack implementation using array
```

Practical 8:- : Implementing a Basic Queue Using Arrays
Problem Statement: Write a menu-driven program to implement a queue using arrays. Include operations like enqueue, dequeue, and display.

```
#include<iostream>
using namespace std;
```

```
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

// Enqueue operation
void enqueue(int value) {
    if (rear == MAX - 1) {
        cout << "Queue Overflow\n";
        return;
    }

    if (front == -1) {
        front = 0;
    }

    rear++;
    queue[rear] = value;
    cout << value << " inserted into queue\n";
}

// Dequeue operation
void dequeue() {
    if (front == -1 || front > rear) {
        cout << "Queue Underflow\n";
        return;
    }

    cout << queue[front] << " removed from queue\n";
    front++;
}

// Display queue
void display() {
if (front == -1 || front > rear) {
    cout << "Queue is empty\n";
    return;
}

cout << "Queue elements: ";
for (int i = front; i <= rear; i++) {
    cout << queue[i]<<" ";
```

```
    }
    cout << endl;
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();

    dequeue();
    display();

    return 0;
}
// queue implementation
```